

Trabalho 1 – Computação de Alto Desempenho

Aluno: Matheus Nunes da Silva (DRE: 125381155) Data: 19/06/2026

Professor: Adriano Côrtes Turma: ECI

1 Exercício 1

1.1 Exercício 1(a)

Abordagem Implementada: Desenvolvimento de um programa introspectivo usando a API MPI associada a chamadas de sistema POSIX/GNU (`getpid` e `sched_getcpu`) para mapear o comportamento do escalonador sobre os núcleos físicos.

Comandos e Saídas: Registros de três execuções consecutivas com sobre-alocação (8 processos).

```
$ mpicc -O2 -Wall -o hello_os hello_os.c -lm
$ mpiexec --oversubscribe -n 8 ./hello_os
Hello from rank 1/8 -- PID = 65048, CPU = 2
Hello from rank 2/8 -- PID = 65049, CPU = 1
Hello from rank 6/8 -- PID = 65058, CPU = 0
Hello from rank 7/8 -- PID = 65061, CPU = 3
Hello from rank 4/8 -- PID = 65055, CPU = 0
Hello from rank 3/8 -- PID = 65050, CPU = 3
Hello from rank 0/8 -- PID = 65047, CPU = 0
Hello from rank 5/8 -- PID = 65056, CPU = 1
$ mpiexec --oversubscribe -n 8 ./hello_os
Hello from rank 1/8 -- PID = 65112, CPU = 0
Hello from rank 2/8 -- PID = 65113, CPU = 2
Hello from rank 3/8 -- PID = 65115, CPU = 3
Hello from rank 0/8 -- PID = 65111, CPU = 0
Hello from rank 5/8 -- PID = 65121, CPU = 3
Hello from rank 4/8 -- PID = 65117, CPU = 2
Hello from rank 7/8 -- PID = 65129, CPU = 2
Hello from rank 6/8 -- PID = 65126, CPU = 3
$ mpiexec --oversubscribe -n 8 ./hello_os
Hello from rank 1/8 -- PID = 65176, CPU = 2
Hello from rank 2/8 -- PID = 65177, CPU = 3
Hello from rank 0/8 -- PID = 65175, CPU = 0
Hello from rank 5/8 -- PID = 65185, CPU = 2
Hello from rank 3/8 -- PID = 65178, CPU = 3
Hello from rank 6/8 -- PID = 65189, CPU = 2
Hello from rank 4/8 -- PID = 65180, CPU = 2
Hello from rank 7/8 -- PID = 65191, CPU = 0
```

Listing 1: Saídas consecutivas do comando `mpiexec` para `hello_os`

Discussão: A análise revela duas propriedades:

1. **PIDs Distintos:** Cada rank MPI opera sob um processo independente do SO, gerando PIDs sequenciais únicos.
2. **Dinamismo na Coluna de CPU:** A mudança na coluna de CPU demonstra a atuação do escalonador Linux. Com `--oversubscribe` ativado em máquina com menos de 8 núcleos, os processos migram dinamicamente ou compartilham a CPU via *time-sharing* para otimizar o balanceamento.

1.2 Exercício 1(b)

Abordagem Implementada: Na variante em anel, calculou-se a topologia usando aritmética modular. Para evitar *deadlocks* devido a chamadas bloqueantes, utilizou-se a primitiva atômica `MPI_Sendrecv`.

```
$ mpicc -O2 -Wall -o hello_ring hello_ring.c -lm
$ mpiexec --oversubscribe -n 4 ./hello_ring
--- Centralizado no Rank 0 ---
Rank 0 recebeu: [Hello de 3]
```

```
Rank 1 recebeu: [Hello de 0]
Rank 2 recebeu: [Hello de 1]
Rank 3 recebeu: [Hello de 2]
```

Listing 2: Resultado da execução centralizada do anel mpi

Discussão dos Parâmetros:

- **Soma de p em source:** O operador % em C preserva o sinal. Ao aplicar $(\text{my_rank} - 1 + p) \% p$, garante-se um operando positivo, retornando com exatidão o rank $p - 1$.
- **Escolha do MPI_Sendrecv:** Se todos executarem MPI_Recv simultaneamente em um anel, ocorre *deadlock*. O MPI_Sendrecv realiza a troca em paralelo de forma não-bloqueante interna.

2 Exercício 2

2.1 Exercício 2(a)

Abordagem e Saída: Desenvolveu-se um bloco para calcular o resto (**rem**) e *offsets* visando dividir a carga de forma assimétrica. A aproximação obtida para $n = 10$ em $f(x) = \sin(x)$ no intervalo $[0, \pi]$ condiz com o valor analítico, sem falha geométrica.

```
$ mpicc -O2 -Wall -o mpi_trap mpi_trap_generalized.c -lm
$ mpiexec --oversubscribe -n 3 ./mpi_trap 10
N = 10 | Resultado: 1.983523537509454 | Tempo: 0.000448 s
```

2.2 Exercício 2(b)

Escalabilidade e Limiar de Eficiência ($E_p < 0.8$):

```
$ mpiexec --oversubscribe -n 1 ./mpi_trap_generalized
N = 10000000 | Resultado: 1.999999999999809 | Tempo: 0.187826 s
$ mpiexec --oversubscribe -n 2 ./mpi_trap_generalized
N = 10000000 | Resultado: 1.999999999999962 | Tempo: 0.213762 s
$ mpiexec --oversubscribe -n 4 ./mpi_trap_generalized
N = 10000000 | Resultado: 2.000000000000020 | Tempo: 0.227560 s
$ mpiexec --oversubscribe -n 8 ./mpi_trap_generalized
N = 10000000 | Resultado: 1.999999999999960 | Tempo: 0.118745 s
```

Listing 3: Execuções de escalabilidade para $n=10^7$

Tabela 1: Métricas de Escalabilidade Forte ($n = 10^7$).

Processos (p)	Tempo T_p (s)	Speedup S_p	Eficiência E_p
1 (T_s)	0.187826	1.0000	1.0000
2	0.213762	0.8786	0.4393
4	0.227560	0.8253	0.2063
8	0.118745	1.5817	0.1977

A eficiência cai drasticamente abaixo de 0.8 **imediatamente a partir de $p = 2$** ($E_2 \approx 0.44$). O problema possui **granularidade extremamente fina**: o processamento de cada trapézio dura microssegundos. Logo, o *overhead* de inicialização de threads MPI e latências de MPI_Reduce superam a paralelização, piorada pelo *oversubscribing*.

2.3 Exercício 2(c)

O erro global da Regra do Trapézio é de ordem $O(h^2)$. Em log-log, a convergência é a inclinação da reta $m \approx 2$. Os dados coletados refinando n com 4 processos paralelos confirmaram isso:

```

$ mpiexec --oversubscribe -n 4 ./mpi_trap_generalized 64
N = 64 | Resultado: 1.999598388656340 | Erro Absoluto: 4.016141e-04

$ mpiexec --oversubscribe -n 4 ./mpi_trap_generalized 256
N = 256 | Resultado: 1.999974900235053 | Erro Absoluto: 2.509976e-05

$ mpiexec --oversubscribe -n 4 ./mpi_trap_generalized 1024
N = 1024 | Resultado: 1.999998431268383 | Erro Absoluto: 1.568732e-06

$ mpiexec --oversubscribe -n 4 ./mpi_trap_generalized 4096
N = 4096 | Resultado: 1.999999901954288 | Erro Absoluto: 9.804571e-08

$ mpiexec --oversubscribe -n 4 ./mpi_trap_generalized 16384
N = 16384 | Resultado: 1.99999993872146 | Erro Absoluto: 6.127854e-09

```

Listing 4: Execuções sucessivas para coleta de dados de convergência

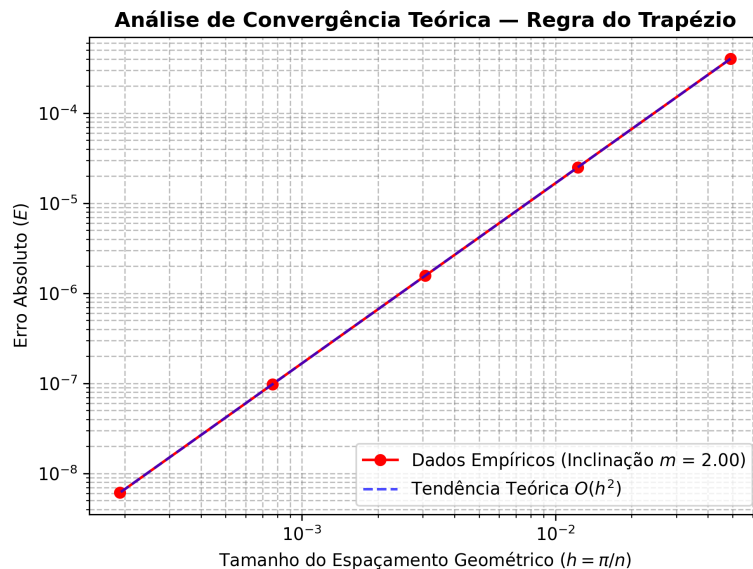


Figura 1: Gráfico Log-Log do erro absoluto versus o tamanho do passo h .

A inclinação analítica calculada foi de **2.00**, validando empiricamente a convergência quadrática assintótica da integração.

3 Exercício 3

Abordagem e Erro Relativo: Distribuição manual de fatias contínuas de 10^5 elementos usando MPI_Send/Recv. O erro da ordem de 10^{-15} comprova a consistência matemática; a divergência decorre da perda de associatividade (IEEE 754) ao somar fora da ordem linear.

```

$ mpiexec --oversubscribe -n 4 ./mpi_psum
Soma Serial: 50055.6838308500
Soma Paralela: 50055.6838308498
Erro Relativo: 4.360718e-15

```

Comparação de Linhas de Código: A abordagem Ponto-a-Ponto (Aula 5) consumiu **32 linhas** no root para gerenciar offsets/loops. A abordagem por Coletivas (Aula 6) reduziu o fluxo para **2 linhas** (MPI_Scatter → MPI_Reduce).

4 Exercício 4

4(a) – MPI_Gather: O Gather simplifica a coleta de strings. O root aloca espaço integrado, recebendo-as ordenadamente sem loops ou sobreposição de memória.

```

$ mpiexec --oversubscribe -n 4 ./mpi_hello_gather
Greetings from process 0 of 4!
Greetings from process 1 of 4!

```

```
Greetings from process 2 of 4!
Greetings from process 3 of 4!
```

4(b) – Min/Max Global: Vetor de 10^5 elementos disperso via `Scatter`. Os extremos foram obtidos via `MPI_Reduce` com `MPI_MIN` e `MPI_MAX`.

```
$ mpiexec --oversubscribe -n 4 ./mpi_minmax
[SERIAL]   Min: 0.0000072047 | Max: 0.9999863814
[PARALELO] Min: 0.0000072047 | Max: 0.9999863814
Verificacao: SUCESSO
```

5 Exercício 5

Para analisar o impacto estrutural da coleta, testaram-se duas variantes de soma de vetores.

```
$ make
mpicc -O2 -Wall -o mpi_vecadd_gather mpi_vecadd_gather.c -lm
mpicc -O2 -Wall -o mpi_vecadd_allgather mpi_vecadd_allgather.c -lm

$ make run
--- Rodando versao GATHER ---
mpiexec --oversubscribe -n 4 ./mpi_vecadd_gather
[GATHER - Processo 0] SUCESSO! Verifica ao serial validada.

--- Rodando versao ALLGATHER ---
mpiexec --oversubscribe -n 4 ./mpi_vecadd_allgather
[ALLGATHER - Processo 3] Verificando ltimo elemento: z_all[15] = 45
```

Listing 5: Execução comparativa das variantes de coleta no terminal

Análise: No `Gather`, o resultado é enviado unicamente à raiz (Processo 0); se outro rank tentasse ler o vetor resultante, causaria falha. No `Allgather`, a primitiva atua combinada, e o Processo 3 validou o resultado com sucesso, provando que todos reconstróem o vetor final sem necessitar de um `MPI_Bcast`.

Quando o `Allgather` é preferível: É a escolha arquitetural correta quando múltiplos (ou todos) os processos necessitam do resultado completo consolidado para dar andamento a computações independentes. Apesar de trafegar mais dados, ele é altamente otimizado na biblioteca (via anéis/hipercubos físicos), possuindo menor latência do que um `Gather` seguido de um `Bcast` manual.

6 Exercício 6

Avaliando a transmissão de *structs* heterogêneas, comparou-se um tipo derivado (`MPI_Type_create_struct`) contra três chamadas separadas de `MPI_Bcast`.

```
$ make run
--- Rodando versao com Tipo Derivado (Struct) ---
mpiexec --oversubscribe -n 4 ./student_struct
[STRUCT - Processo 0] Transmitindo: 'Ada Lovelace', Nota: 9.8, ID: 101010
[STRUCT - Processo 1] Recebeu: 'Ada Lovelace', Nota: 9.8, ID: 101010
[STRUCT - Processo 2] Recebeu: 'Ada Lovelace', Nota: 9.8, ID: 101010
[STRUCT - Processo 3] Recebeu: 'Ada Lovelace', Nota: 9.8, ID: 101010

--- Rodando versao com 3 Bcasts separados ---
mpiexec --oversubscribe -n 4 ./student_three_bcasts
[3 BCASTS - Processo 0] Transmitindo: 'Ada Lovelace', Nota: 9.8, ID: 101010
[3 BCASTS - Processo 1] Recebeu: 'Ada Lovelace', Nota: 9.8, ID: 101010
[3 BCASTS - Processo 2] Recebeu: 'Ada Lovelace', Nota: 9.8, ID: 101010
[3 BCASTS - Processo 3] Recebeu: 'Ada Lovelace', Nota: 9.8, ID: 101010
```

Discussão:

- Equivalência e Volume:** Logicamente os resultados são idênticos, porém a versão derivada resolve o envio em 1 transação de rede, contra 3 da estratégia separada.
- Impacto na Latência:** Toda comunicação possui um *overhead* fixo (empacotamento, cabeçalhos, sincronização). Disparar três chamadas significa pagar esse custo triplicado. O tipo derivado compacta o *layout* na memória local e paga a latência de rede apenas uma vez.

Conclusão: Transmissões fragmentadas tornam o sistema gargalado pela rede (*latency-bound*). A consolidação em tipos derivados é obrigatória para aplicações de alta escalabilidade.